

The Producer/consumer (Bounded buffer) problem

↳ first proposed by Dijkstra

Imagine one or more producer threads and one or more consumer threads.

Producers → generate data items and place them in a buffer

Consumers → grab said items from the buffer and consume in some way.

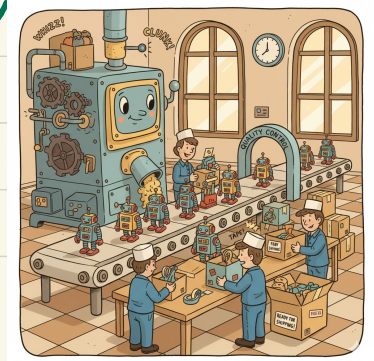
For eg:- ① consider web servers/request handling.

- Producer → incoming HTTP requests from clients
- consumer → worker threads that process requests.
- Buffer → request queue.

A coordination is required such that requests aren't lost, workers aren't idle if requests are available, queue doesn't grow infinitely under high load.

② consider Assembly line/factory production

- Producer → machines assembling parts



- consumer → next stage in assembly line or packaging system

- Buffer → conveyor belt or storage b/w stages

co-ordination ensures no machine overload and no idle time for downstream processes.

In general, a producer-consumer pattern appears whenever you have a buffer, concurrent data generation, and asynchronous consumption.

Another familiar example where a bounded buffer is used is when you pipe the output of one program to another.

eg:- `grep foo file.txt | wc -l`

- runs two processes concurrently; `grep` writes lines from `file.txt` with the string `foo` in them to what it thinks is standard input.

What is the buffer here? → UNIX pipe (created by `pipe()` system call)

The other end of the pipe is connected to the standard I/P of the process `wc`, that simply counts the #lines in the I/P stream and prints out the result.

- ∴ producer → grep process
consumer → wc process.

```
1 int buffer;  
2 int count = 0; // initially, empty  
3  
4 void put(int value) {  
5     assert(count == 0);  
6     count = 1;  
7     buffer = value;  
8 }  
9  
10 int get() {  
11     assert(count == 1);  
12     count = 0;  
13     return buffer;  
14 }
```

→ two routines to **put** a value into the shared buffer and to **get** a value out of the buffer

(example that has a single entry in the buffer → will generalize later).

Goal: write routines that know when it is ok to access the buffer to either **put** data into it or **get** data out of it.

when buffer size is 1 (single slot)

"only put data into the buffer when **count** is **zero** (i.e., when buffer is empty) and only get data from the buffer when **count** is **one** (i.e., when buffer is full)

Scenario 1: single producer and single consumer ; buffer size : 1

How to use the above put() and get() routines (without race conditions) to solve the problem in the above scenario.

```
1  int loops; // must initialize somewhere...
2  cond_t cond;
3  mutex_t mutex;
4
5  void *producer(void *arg) {
6      int i;
7      for (i = 0; i < loops; i++) {
8          Pthread_mutex_lock(&mutex);
9          if (count == 1)
10             Pthread_cond_wait(&cond, &mutex);
11          put(i);
12          Pthread_cond_signal(&cond);
13          Pthread_mutex_unlock(&mutex);
14      }
15  }
```

Producer thread

```
16
17  void *consumer(void *arg) {
18      int i;
19      for (i = 0; i < loops; i++) {
20          Pthread_mutex_lock(&mutex);
21          if (count == 0)
22             Pthread_cond_wait(&cond, &mutex);
23          int tmp = get();
24          Pthread_cond_signal(&cond);
25          Pthread_mutex_unlock(&mutex);
26          printf("%d\n", tmp);
27      }
28  }
```

Consumer thread

→ indicates how many times the producer should produce and how many items the consumer should consume

// p1 → acquire lock
// p2 → buffer full?
// p3 → wait until consumer signals
// p4 → put an item in the buffer
// p5 → wake consumer (item available)
// p6 → release lock.

// c1 → acquire lock
// c2 → buffer empty?
// c3 → wait for producer
// c4 → remove item from buffer
// c5 → wake producer (space available)
// c6 → release the lock

The above code works perfectly in scenario 1 (1 producer and 1 consumer)

But, can we use it if there are multiple producers or consumers? **NO!**

Problem 1: the if statement before the wait.

A possible thread race with 2 consumers (T_{c1} and T_{c2}) and 1 producer (T_p)

T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
if (Count==0) returns true	c1 Run	Ready	Ready	Ready	Ready	0	
	c2 Run	Ready	Ready	Ready	Ready	0	
	c3 Sleep	Ready	Ready	Ready	Ready	0	Nothing to get
	Sleep	Ready	Ready	p1 Run	Run	0	if (Count==1) is false so proceed to
	Sleep	Ready	Ready	p2 Run	Run	0	
	Sleep	Ready	Ready	p4 Run	Run	1	Buffer now full adding the first item (P2)
	Ready	Ready	Ready	p5 Run	Run	1	T_{c1} awoken
	Ready	Ready	Ready	p6 Run	Run	1	issue signal (only T_{c1} was in queue, and it is ready now)
	Ready	Ready	Ready	p1 Run	Run	1	
	Ready	Ready	Ready	p2 Run	Run	1	if (Count==1) is true
if (Count==0) returns false	Ready	Ready	Ready	p3 Sleep	Sleep	1	Buffer full; sleep
proceed to get the item	Ready	c1 Run	Run	Sleep	Sleep	1	T_{c2} sneaks in ...
issues signal (only T_p in queue)	Ready	c2 Run	Run	Sleep	Sleep	1	problem!
	Ready	c4 Run	Run	Sleep	Sleep	0	... and grabs data
	Ready	c5 Run	Run	Ready	Ready	0	T_p awoken
	Ready	c6 Run	Run	Ready	Ready	0	
	c4 Run	Ready	Ready	Ready	Ready	0	Oh oh! No data

didn't check the condition again as we used if instead of while!

How to fix the above problem? use while instead of if.
(think why it works now!)

Are there still problems in the code? Unfortunately, YES!

↳ there is only one condition variable for both producers and consumers!

What is the problem with this?

↳ at some point the following scenario might happen.

- both T_{c1} and T_{c2} go to sleep (upon executing c3)
- T_p adds value into buffer → wake one of the consumers (say T_{c1})
→ loops again → see buffer full → go to sleep

// threads sleeping in the queue → T_{c2}, T_p .
where as T_{c1} is ready to run

- T_{c1} rechecks the condition → buffer full, so consumes it and issue a signal to wake someone in the queue.

Problem? How do we know whom to wake up?

Bad case: wakes T_{c2} → T_{c1} go to sleep.

T_{c2} runs $\rightarrow$ recheck condition $\rightarrow$ see buffer is empty $\rightarrow$ go back to sleep.
 All threads are sleeping now!
 $\rightarrow$ a clear bug!

T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
c1	Run		Ready		Ready	0	
c2	Run		Ready		Ready	0	
c3	Sleep		Ready		Ready	0	Nothing to get
	Sleep	c1	Run		Ready	0	
	Sleep	c2	Run		Ready	0	
	Sleep	c3	Sleep		Ready	0	Nothing to get
	Sleep		Sleep	p1	Run	0	
	Sleep		Sleep	p2	Run	0	
	Sleep		Sleep	p4	Run	1	Buffer now full
	Ready		Sleep	p5	Run	1	T_{c1} awoken
	Ready		Sleep	p6	Run	1	
	Ready		Sleep	p1	Run	1	
	Ready		Sleep	p2	Run	1	
	Ready		Sleep	p3	Sleep	1	Must sleep (full)
c2	Run		Sleep		Sleep	1	Recheck condition
c4	Run		Sleep		Sleep	0	T_{c1} grabs data
c5	Run		Ready		Sleep	0	Oops! Woke T_{c2}
c6	Run		Ready		Sleep	0	
c1	Run		Ready		Sleep	0	
c2	Run		Ready		Sleep	0	
c3	Sleep		Ready		Sleep	0	Nothing to get
	Sleep	c2	Run		Sleep	0	
	Sleep	c3	Sleep		Sleep	0	Everyone asleep...

What exactly created the problem? → a consumer waking up another consumer!

How to solve the above problem?

→ use two condition variables; one for producers and one for consumers

```
1  cond_t empty, fill;
2  mutex_t mutex;
3
4  void *producer(void *arg) {
5      int i;
6      for (i = 0; i < loops; i++) {
7          Pthread_mutex_lock(&mutex);           // p1
8          while (count == 1)                    // p2
9              Pthread_cond_wait(&empty, &mutex); // p3
10         put(i);                               // p4
11         Pthread_cond_signal(&fill);           // p5
12         Pthread_mutex_unlock(&mutex);         // p6
13     }
14 }
15
16 void *consumer(void *arg) {
17     int i;
18     for (i = 0; i < loops; i++) {
19         Pthread_mutex_lock(&mutex);           // c1
20         while (count == 0)                    // c2
21             Pthread_cond_wait(&fill, &mutex); // c3
22         int tmp = get();                      // c4
23         Pthread_cond_signal(&empty);          // c5
24         Pthread_mutex_unlock(&mutex);         // c6
25         printf("%d\n", tmp);
26     }
27 }
```

→ waits on empty and signals fill

→ a correct
Producer/consumer
synchronization

→ waits on fill signals empty.

what if we need more buffer slots?

```
1 int buffer[MAX]; → shared bdd array
2 int fill_ptr = 0; → index to put next item
3 int use_ptr = 0; → index to get next item
4 int count = 0; → #filled slots
5
6 void put(int value) {
7     buffer[fill_ptr] = value;
8     fill_ptr = (fill_ptr + 1) % MAX;
9     count++;
10 }
11
12 int get() {
13     int tmp = buffer[use_ptr];
14     use_ptr = (use_ptr + 1) % MAX;
15     count--;
16     return tmp;
17 }
```

modified put() and get() functions

```
1 cond_t empty, fill;
2 mutex_t mutex;
3
4 void *producer(void *arg) {
5     int i;
6     for (i = 0; i < loops; i++) {
7         Pthread_mutex_lock(&mutex); // p1
8         while (count == MAX) // p2
9             Pthread_cond_wait(&empty, &mutex); // p3
10        put(i); // p4
11        Pthread_cond_signal(&fill); // p5
12        Pthread_mutex_unlock(&mutex); // p6
13    }
14 }
15
16 void *consumer(void *arg) {
17     int i;
18     for (i = 0; i < loops; i++) {
19         Pthread_mutex_lock(&mutex); // c1
20         while (count == 0) // c2
21             Pthread_cond_wait(&fill, &mutex); // c3
22         int tmp = get(); // c4
23         Pthread_cond_signal(&empty); // c5
24         Pthread_mutex_unlock(&mutex); // c6
25         printf("%d\n", tmp);
26     }
27 }
```

a slight modification in the while condition of producer thread.

Another problem where the thread to be waken up becomes crucial.
→ multi-threaded memory allocation library.

```
1 // how many bytes of the heap are free?
2 int bytesLeft = MAX_HEAP_SIZE;
3
4 // need lock and condition too
5 cond_t c;
6 mutex_t m;
7
8 void *
9 allocate(int size) {
10     Pthread_mutex_lock(&m);
11     while (bytesLeft < size)
12         Pthread_cond_wait(&c, &m);
13     void *ptr = ...; // get mem from heap
14     bytesLeft -= size;
15     Pthread_mutex_unlock(&m);
16     return ptr;
17 }
18
19 void free(void *ptr, int size) {
20     Pthread_mutex_lock(&m);
21     bytesLeft += size;
22     Pthread_cond_signal(&c); // whom to signal??
23     Pthread_mutex_unlock(&m);
24 }
```

why this is a problem here?
consider the foll. scenario
at some point of time.

- zero bytes are free.
- thread T_a calls `allocate(100)`
→ checks condition → True
→ cond-wait → go to sleep.

- thread T_b calls `allocate(10)` → go to sleep
as above

- thread T_c calls `free(50)`
→ issue signal

Problem: if thread T_a wakes up!
(think, why?) → a bug!

How to solve this? → replace `pthread_cond_signal`
by `pthread_cond_broadcast()` → wakes up all waiting threads.
"covering condition"

Drawback → negative impact on performance (think, why?)